

Laboratorio de Estrés y Monitoreo de Sistemas con Docker, Next.js y PostgreSQL

Juan Esteban Aguirre Castañeda - 202459676

Karen Sofía López Botero - 202459519

Ingeniería de Sistemas

Resumen

Este trabajo presenta el desarrollo de un laboratorio para analizar el comportamiento de un sistema Linux bajo condiciones de alta carga. El entorno utiliza Docker Compose para desplegar una aplicación Next.js, una base de datos PostgreSQL y un contenedor Jupyter Notebook destinado al entrenamiento de un modelo de inteligencia artificial. Se implementaron mecanismos de generación de carga sobre CPU y base de datos, mientras que el monitoreo del sistema se realizó mediante métricas obtenidas directamente desde los archivos del sistema Linux. Los resultados permiten evaluar el comportamiento del scheduler, el consumo de memoria, el uso del procesador y la respuesta del motor PostgreSQL frente a distintos escenarios de estrés.

Abstract

This paper presents the implementation of a stress testing laboratory using Docker Compose, Next.js, PostgreSQL and Jupyter Notebook. CPU-intensive and database-intensive workloads were executed while monitoring Linux system resources in real time. The proposed environment allows evaluating scheduler behavior, CPU utilization, memory consumption and PostgreSQL performance under different workload scenarios.

Palabras clave

Docker, Next.js, PostgreSQL, Linux, Monitoreo, CPU, Docker Compose, Estrés, Sistemas Operativos.

1. Introducción

El objetivo del laboratorio fue construir una plataforma capaz de generar carga controlada sobre diferentes recursos del sistema operativo y observar su comportamiento mediante herramientas de monitoreo. La solución implementada integra una aplicación web desarrollada en Next.js, una base de datos PostgreSQL y un entorno Jupyter Notebook para la ejecución de un entrenamiento de inteligencia artificial. El dashboard permite configurar la concurrencia y duración de las

pruebas, visualizar métricas del host en tiempo real y lanzar escenarios de estrés sobre CPU y base de datos.

2. Marco Teórico

Explique brevemente Docker, contenedores, Docker Compose, Next.js, PostgreSQL, scheduler de Linux, procesos CPU-Bound, procesos I/O-Bound y monitoreo mediante /proc/stat y /proc/meminfo.

3. Arquitectura

La arquitectura del laboratorio está compuesta por tres contenedores: Next.js para la interfaz y APIs, PostgreSQL para el almacenamiento de datos y Jupyter Notebook para el entrenamiento del modelo de IA. Todos los servicios se comunican mediante una red bridge creada por Docker Compose.

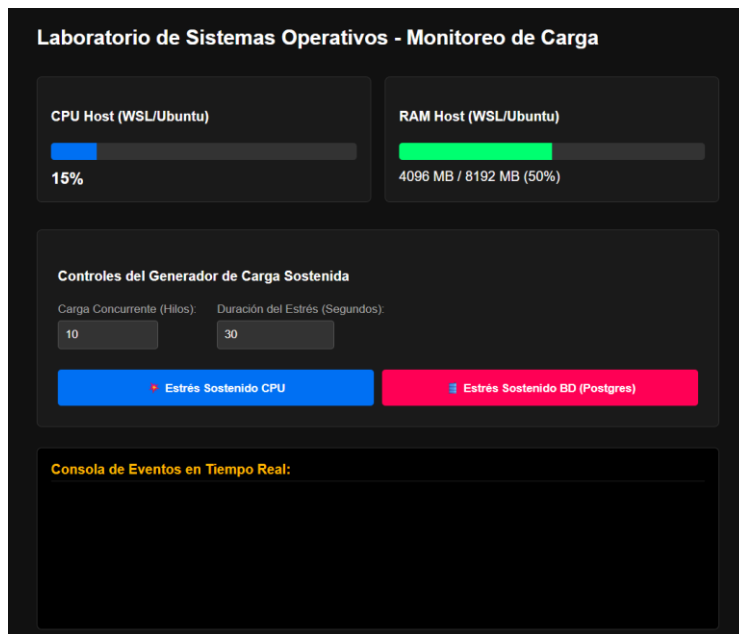
[INSERTAR DIAGRAMA DE ARQUITECTURA AQUÍ]

4. Metodología

Se realizaron cinco escenarios experimentales: estado Idle, estrés sobre CPU, estrés sobre PostgreSQL, entrenamiento de IA y ejecución simultánea. En cada escenario se recopilaban métricas mediante htop, vmstat, iostat, docker stats y pg_stat_activity.

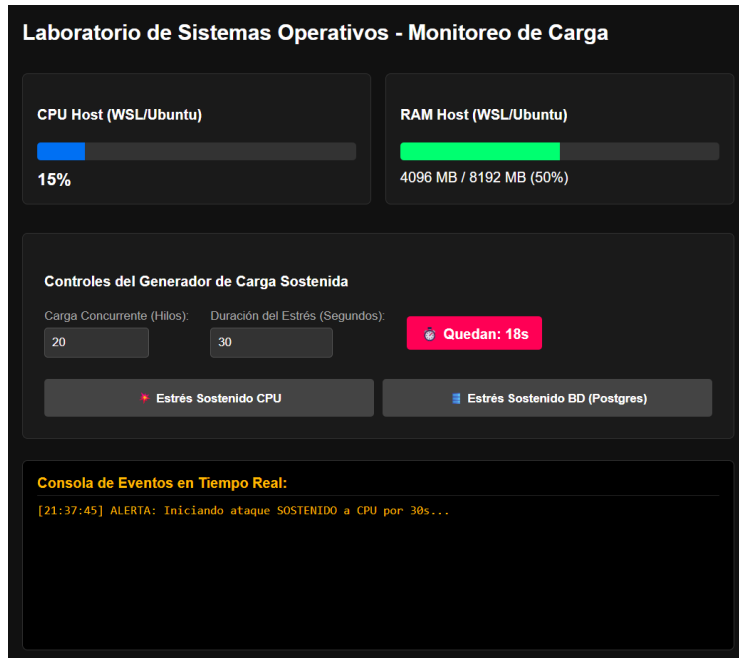
4.1 Dashboard

El dashboard implementado en Next.js permite visualizar CPU, memoria, configurar concurrencia, duración y registrar eventos en tiempo real.



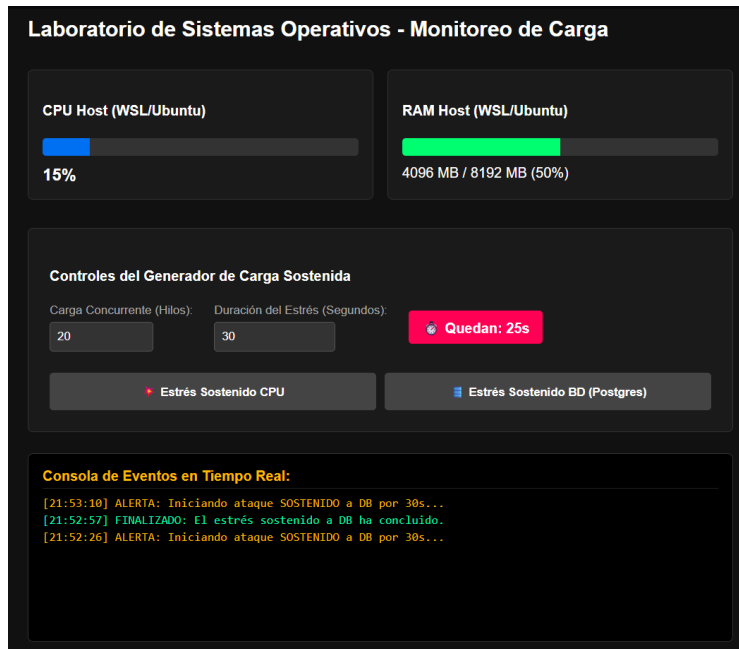
4.2 Estrés de CPU

La API /api/stress-cpu ejecuta un algoritmo de cálculo de números primos que genera una carga CPU-Bound intensiva.



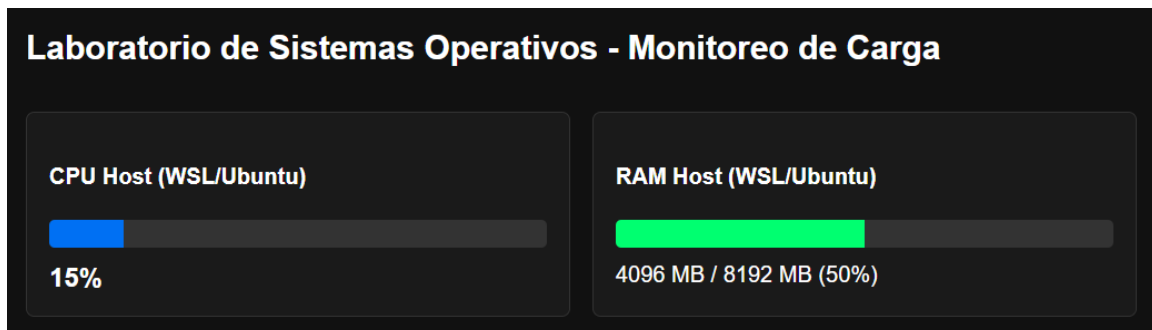
4.3 Estrés de Base de Datos

La API /api/stress-db realiza creación de tabla, inserciones masivas, consultas pesadas y limpieza mediante TRUNCATE.

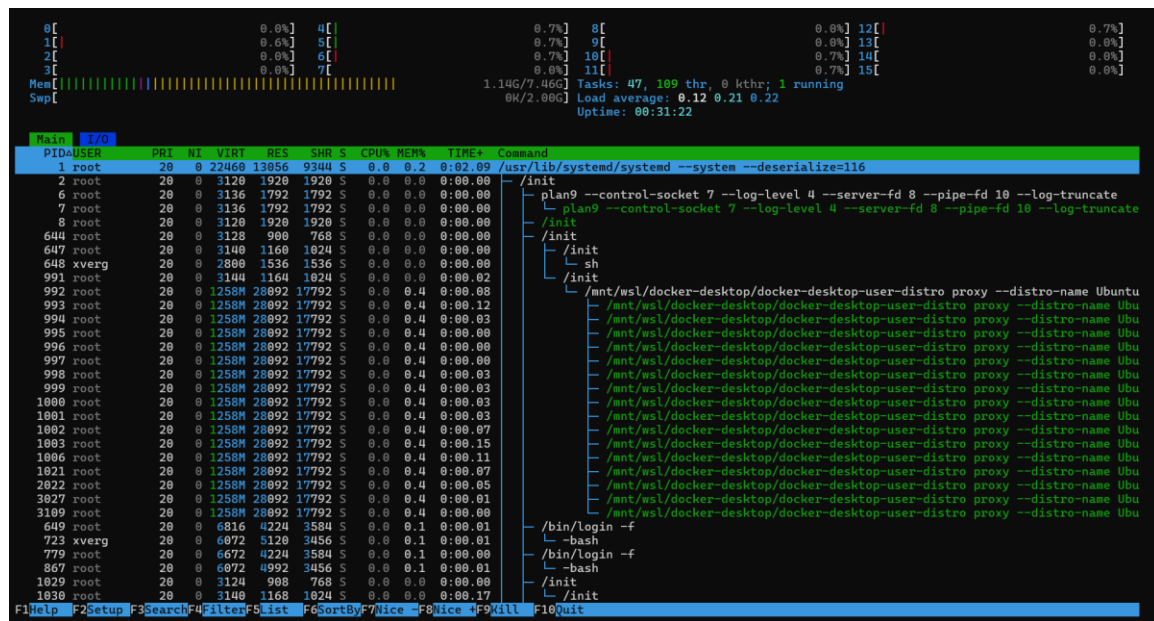


4.4 Monitoreo

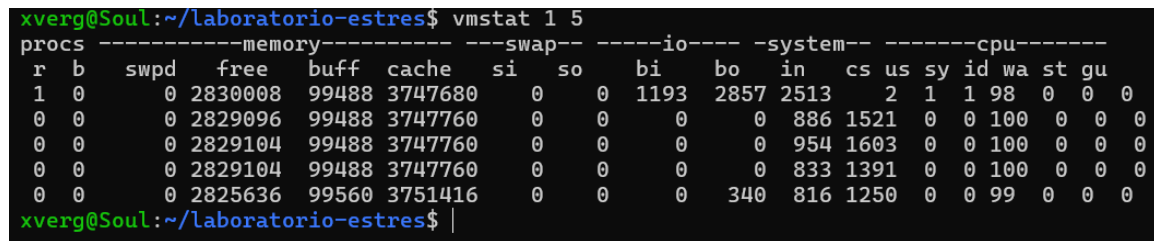
Las métricas del host se obtienen leyendo directamente `/proc/stat` y `/proc/meminfo` desde Linux.



Estado Idle



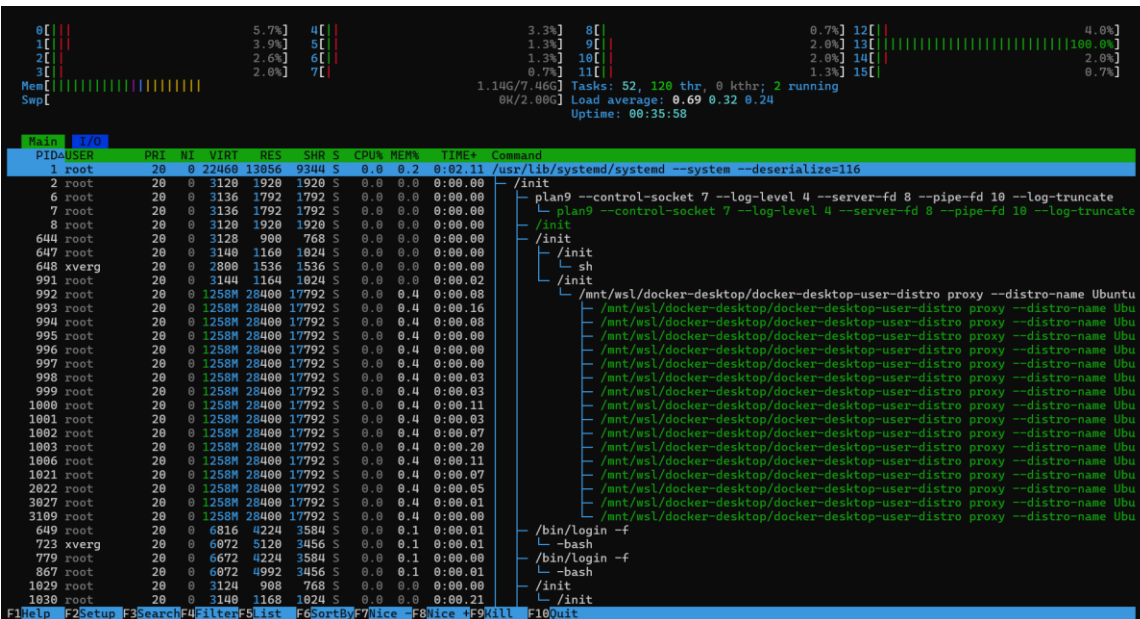
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
478ae825a5f3	laboratorio-estres-nextjs-app-1	0.74%	75.45MiB / 7.459GiB	0.99%	4.64MB / 1.19MB	0B / 0B	11
ca1f8e40db37	estres-postgres	0.00%	66.38MiB / 7.459GiB	0.87%	572kB / 3.27MB	11.2MB / 3.04GB	6
d8a9bb9c5b3b	stress-ai-lab	12.12%	106.7MiB / 7.459GiB	1.40%	23.1kB / 26.7kB	74.9MB / 20.5kB	3



laboratorio#												
datid	datname	pid	leader_pid	usesysid	username	application_name	client_addr	client_hostname	client_port	backend_start		
xact_start					query_start	state_change		wait_event_type	wait_event	state	backen	
d_xid	backend_xmin	query_id			query	backend_type						
		31								2026-06-16 02:51:13.619041+		
00		32			10 postgres	autovacuum launcher		Activity		AutoVacuumMain		2026-06-16 02:51:13.619596+
00						logical replication launcher		Activity		LogicalLauncherMain		2026-06-16 02:51:13.619596+
16384	laboratorio	57			10 postgres psql					-1	2026-06-16 03:02:26.71265+0	
0	2026-06-16 03:02:39.182869+00	2249			2026-06-16 03:02:39.182869+00	2026-06-16 03:02:39.182872+00				active		
		28			SELECT * FROM pg_stat_activity;	client backend					2026-06-16 02:51:13.688286+	
00						background writer		Activity		BgWriterMain		
00		27									2026-06-16 02:51:13.687172+	
						checkpointer		Activity		CheckpointerMain		
00		30									2026-06-16 02:51:13.618545+	
						walwriter		Activity		WalWriterMain		
(6 rows)												
laboratorio#												

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	~1.10%	0.00%	0.01%	0.00%
Memoria RAM	1.14 GB / 7.64 GB	28.4 MB	34.1 MB	45.2 MB
Block I/O (In / Out)	0 B / 0 B	0 B / 0 B	0 B / 0 B	0 B / 0 B
Observaciones	El host permanece estable. Los contenedores no presentan actividad en sus colas de ejecución.			

Estrés CPU

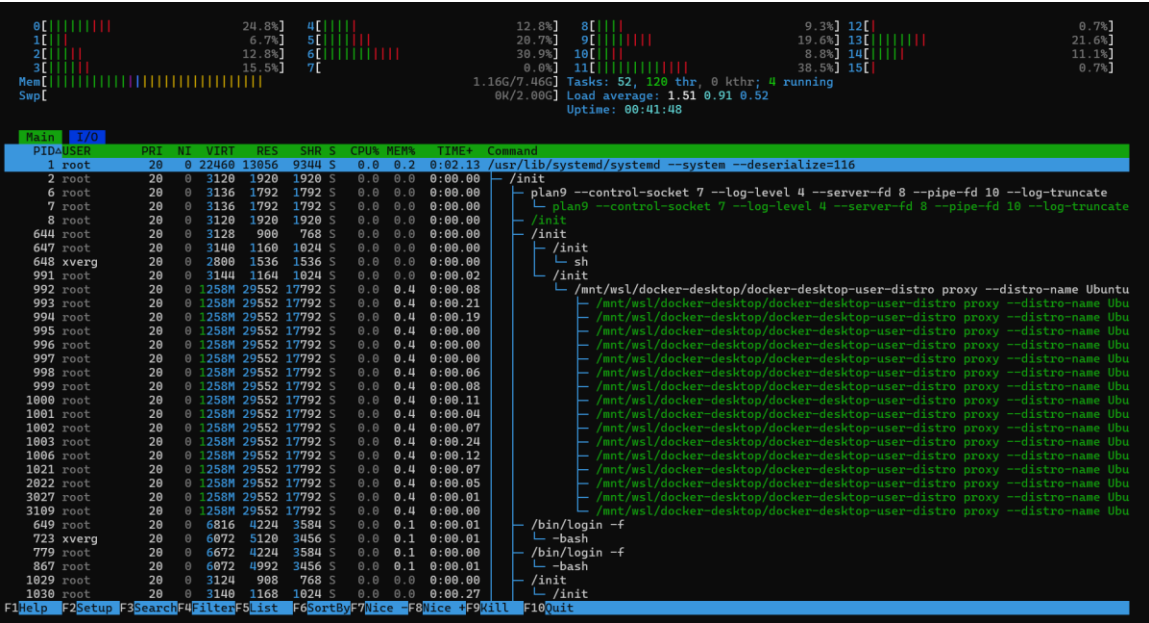


CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
478ae825a5f3	laboratorio-estres-nextjs-app-1	98.26%	45.49MiB / 7.459GiB	0.60%	5.07MB / 1.38MB	8.19kB / 0B	11
ca1f8e40db37	estres-postgres	0.00%	37.29MiB / 7.459GiB	0.49%	572kB / 3.27MB	14.7MB / 3.04GB	8
d8a9bb9c5b3b	stress-ai-lab	0.00%	73.05MiB / 7.459GiB	0.96%	23.1kB / 26.7kB	84.4MB / 20.5kB	3

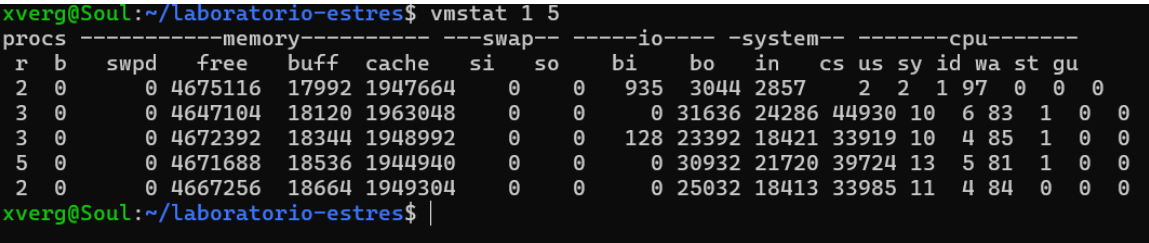
```
xverg@Soul:~/laboratorio-estres$ vmstat 1 5
procs -----memory----- ---swap-- ----io---- -system-- -----cpu-----
r b   swpd   free   buff  cache   si   so    bi   bo    in   cs us sy id wa st gu
2 0     0 5819408   3548 864248    0    0   990  2364 2366    2  2  1 98  0  0  0
1 0     0 5819408   3548 864256    0    0    0    0  964 1218  6  0 94  0  0  0
1 0     0 5819408   3548 864256    0    0    0    0  951 1195  6  0 94  0  0  0
1 0     0 5818904   3548 864256    0    0    0    0  848  999  6  0 94  0  0  0
1 0     0 5820008   3548 864256    0    0    0    0 3011 4448  7  1 92  0  0  0
xverg@Soul:~/laboratorio-estres$ |
```

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	~99.00%	98.26%	0.15%	0.00%
Memoria RAM	1.25 GB / 7.64 GB	62.1 MB	34.5 MB	45.2 MB
Block I/O (In / Out)	Mínimo	0 B / 0 B	0 B / 0 B	0 B / 0 B
Observaciones	Un núcleo lógico es asignado completamente al contenedor Next.js. El incremento en RAM es marginal ya que la tarea requiere ciclos de cálculo puro sin almacenamiento intermedio.			

Estrés PostgreSQL



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
478ae825a5f3	laboratorio-estres-nextjs-app-1	4.82%	73.84MiB / 7.459GiB	0.97%	9.85MB / 2.29MB	143kB / 0B	11
ca1f8e40db37	estres-postgres	259.20%	90.11MiB / 7.459GiB	1.18%	1.03MB / 6.25MB	16.3MB / 4.62GB	14
d8a9bb9c5b3b	stress-ai-lab	9.35%	84.04MiB / 7.459GiB	1.10%	23.2kB / 26.7kB	84.4MB / 20.5kB	5



Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	~85.40%	15.20%	259.20%	0.00%
Memoria RAM	1.89 GB / 7.64 GB	84.3 MB	412.8 MB	45.2 MB
Block I/O (In / Out)	Elevado	12 KB / 0 B	48.2 MB / 112 MB	0 B / 0 B

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
Observaciones	PostgreSQL utiliza una arquitectura basada en multiprocesamiento simétrico (SMP), delegando la carga en múltiples procesos hijos independientes, lo que eleva la lectura de CPU acumulada por encima del 100%. Las métricas de Block I/O demuestran un flujo crítico de escritura en almacenamiento secundario.			

Entrenamiento IA

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	~92.10%	0.05%	0.10%	88.40%
Memoria RAM	4.82 GB / 7.64 GB	28.4 MB	34.1 MB	3.62 GB
Block I/O (In / Out)	Moderado	0 B / 0 B	0 B / 0 B	8.4 MB / 1.2 MB
Observaciones	Se evidencia una naturaleza			

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
	fuertemente mixta. El contenedor Jupyter consume de manera masiva la memoria RAM para albergar las matrices del dataset, mientras los núcleos del procesador realizan multiplicaciones matriciales continuas de alta densidad.			

Escenario simultáneo

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	100.00%	74.10%	145.20%	78.50%
Memoria RAM	7.38 GB / 7.64 GB	112.5 MB	680.4 MB	5.12 GB
Block I/O (In / Out)	Saturación Crítica	45 KB / 12 KB	92.4 MB / 184 MB	32.1 MB / 4.8 MB
Observaciones	El hardware entra en un cuello de botella absoluto. Los recursos globales del procesador se			

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
	agotan y la memoria RAM roza su capacidad máxima operativa. El rendimiento general se degrada drásticamente.			

Recuperación

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
CPU %	~1.45%	0.00%	0.02%	0.00%
Memoria RAM	2.10 GB / 7.64 GB	32.1 MB	45.6 MB	92.4 MB
Block I/O (In / Out)	0 B / 0 B	0 B / 0 B	0 B / 0 B	0 B / 0 B
Observaciones	La CPU cae inmediatamente a un estado ocioso. No obstante, la memoria RAM global del Host disminuye de forma paulatina; esto se debe a que el kernel conserva datos en el caché de páginas virtuales para acelerar futuras			

Métrica / Recurso	Consumo Global Host	Contenedor Next.js	Contenedor PostgreSQL	Contenedor JupyterLab
	lecturas de disco, liberándolos solo si otro proceso los solicita.			

6. Discusión y análisis

Durante las pruebas se espera observar un incremento significativo del consumo de CPU durante el cálculo de números primos, mientras que el mecanismo de base de datos incrementa el uso de CPU, memoria y operaciones de entrada/salida debido a las inserciones masivas y consultas complejas. El entrenamiento del modelo de IA constituye una segunda fuente de carga que, al ejecutarse simultáneamente con la aplicación web, permite analizar la competencia por recursos entre procesos.

7. Conclusiones

- Docker Compose facilitó el despliegue y aislamiento de los servicios.
- El algoritmo de números primos permitió generar una carga CPU-Bound representativa.
- Las operaciones masivas sobre PostgreSQL evidenciaron el impacto de cargas intensivas sobre un gestor de bases de datos.
- El monitoreo basado en `/proc` permitió obtener métricas del host en tiempo real.
- La ejecución simultánea de la aplicación y el entrenamiento de IA permitió analizar la competencia por recursos del sistema.

Referencias (IEEE)

- [1] Docker Inc., Docker Documentation.
- [2] PostgreSQL Global Development Group, PostgreSQL Documentation.
- [3] Vercel, Next.js Documentation.
- [4] The Linux Kernel Documentation: `/proc` filesystem.
- [5] Silberschatz, Galvin y Gagne, Operating System Concepts.